SMART PARKING SYSTEM

QA Backend Function Testing & Reporting

Document Status: Draft – Team Review

Test Environment: Local Development Environment

Evidence Location: `.cache/qa-run`

Report Date: 12 October 2026

Tester: Hoàng Phan

1. Objective

The objective of this QA activity is to evaluate the available backend function-test evidence for the Smart Parking System, identify authentication and authorization issues, record test limitations, and report which functional workflows have and have not been verified.

The report is based on the recorded automated test summaries, HTTP test results, and ParkingService investigation notes. It does not claim complete acceptance-test coverage.

2. Scope

The test scope covers the following functional areas:

- **SPARK-177 – Login/Logout:** Authentication, protected endpoints, logout, and session revocation.
- **SPARK-181 – Driver/Owner Registration:** Registration request validation and the available registration workflow evidence.
- **SPARK-197 – Admin Approve Owner Account:** Admin access to Owner applications and the available approval workflow evidence.
- **SPARK-182 – Parking Structure Management:** Parking-lot access control, structure retrieval, and operational-layout investigation.

The existing Excel Login/Logout sheets are retained as historical test records. Their results must be distinguished from the current QA run where test dates, service instances, or observed HTTP responses differ.

3. Test Environment and Evidence

The recorded environment includes the local UserService and ParkingService instances. The evidence files are stored under:

`.cache/qa-run`

The available evidence includes:

- `backend-function-test-results.csv`
- `positive-auth-results.csv`
- `parking-function-batch.csv`
- `dotnet-test-summary.csv`
- UserService and ParkingService automated test logs

The report uses only the results recorded in these artifacts and the accompanying test investigation notes.

4. Automated Test Summary

Test Project	Total	Passed	Skipped	Failed
UserService	45	40	5	0
ParkingService	13	13	0	0
Total	**58**	**53**	**5**	**0**

The recorded automated test runs reported 53 passed tests, 5 skipped tests, and no failed tests.

Five database-dependent tests were skipped because `SMARTPARK_AUTH_TEST_CONNECTION` was not configured for an isolated PostgreSQL test database. These tests are not counted as passed.

The absence of automated test failures does not establish that every backend feature or business workflow has been verified.

5. HTTP Function-Test Summary

The current evidence records 16 HTTP checks:

- 12 negative HTTP checks.
- 4 positive authentication/Admin checks.

Selected observed results include:

- An unauthenticated request to `GET /api/auth/me` returned HTTP 401.
- Empty login, Driver registration, and Owner registration requests returned HTTP 400.
- An authenticated profile request returned HTTP 200.
- The Owner-application list returned HTTP 200 with an empty data collection.
- A recorded logout request returned HTTP 200.
- Reusing the access token after logout returned HTTP 401.

These results support the specific authentication, validation, and access-control checks performed. They do not establish complete registration, approval, or parking-structure functionality.

6. Driver and Owner Registration

Verified Checks

- Submitting an empty Driver registration request returned HTTP 400 Bad Request.
- Submitting an empty Owner registration request returned HTTP 400 Bad Request.

These results confirm that the tested empty requests were rejected.

Outstanding Verification

A successful Driver registration flow was not established by the current evidence.

The complete Owner registration lifecycle—including valid registration, contact verification, pending approval state, and subsequent account-state verification—was also not demonstrated end to end.

Therefore, the registration feature must not be reported as fully passed based solely on the negative validation checks.

7. Admin Owner Approval

Verified Checks

- An unauthenticated request to list Owner applications returned HTTP 401 Unauthorized.
- An unauthenticated Owner-application review request returned HTTP 401 Unauthorized.
- An authenticated Admin request to list Owner applications returned HTTP 200.

The Owner-application list was empty at the time of the recorded test.

Outstanding Verification

A successful approval was not demonstrated because no eligible Owner application was available in the recorded test context.

The current evidence does not establish that an authorized Admin can successfully approve a verified pending Owner application and that the account transitions to the expected post-approval state.

The Admin approval workflow remains unverified end to end.

8. Parking Structure Management

Verified Checks

- An unauthenticated request to `GET /api/parking-lots` returned HTTP 401 Unauthorized.
- An unauthenticated request to `POST /api/parking-lots` returned HTTP 401 Unauthorized.
- An unauthenticated request to `GET /api/parking-lots/{id}/structure` returned HTTP 401 Unauthorized.
- A request to `GET /api/parking-lots` using the tested Admin access token returned HTTP 401 Unauthorized.

The unauthenticated responses support the expected rejection of requests without authentication. The Admin-token result does not validate the Owner workflow because the tested identity was not an eligible Owner identity for the required Owner scope.

Blocked or Inconclusive Checks

The positive parking-structure and operational-layout workflows were blocked because a valid Owner scope and parking-lot context were not available.

Two conventional OpenAPI discovery paths, `/openapi/v1.json` and `/openapi.json`, returned HTTP 404. This is inconclusive and does not by itself prove that the ParkingService API is broken.

A successful Owner workflow for creating a parking lot, retrieving its structure, and retrieving its operational layout has not been established by the current evidence.

9. Test Limitations

The following limitations affect the current test coverage:

1. Five database-dependent automated tests were skipped because the required isolated PostgreSQL test connection was not configured.
2. A valid verified Owner account and the required Owner permissions were not available for the positive ParkingService workflow.
3. No eligible Owner application was available to verify successful Admin approval.
4. Registration, verification, and approval were not tested as a complete end-to-end lifecycle.
5. HTTP status checks alone do not prove business-rule correctness, database persistence, or complete feature behavior.
6. Existing Login/Logout workbook results originate from earlier test records and must not be silently merged with current results when the observed responses differ.

10. Recommended Follow-up Actions

1. Configure a dedicated isolated PostgreSQL test database and execute the five skipped automated tests.
2. Prepare a valid Owner test account with completed contact verification and the required parking permissions.
3. Verify that ParkingService communicates with the intended UserService instance.
4. Execute the positive Parking Structure Management workflow: create a parking lot, retrieve its structure, retrieve its operational layout, and verify persisted data.

5. Execute Owner registration, contact verification, Admin review/approval, and post-approval access as an end-to-end workflow.
6. Capture request/response evidence and map each executed test case to its Jira issue and acceptance criteria.
7. Update the Excel summary formulas to include the newly added Registration, Admin Approval, and Parking Structure Management sheets.

11. QA Conclusion

The recorded automated test suites reported **53 passed, 5 skipped, and 0 failed**. Selected HTTP checks also demonstrated expected authentication enforcement and request validation behavior.

However, successful Driver/Owner registration, Admin approval, and the positive Owner parking-structure workflow remain unverified end to end. The skipped database-dependent tests also require follow-up.

Final Assessment: Partial QA evidence available. Further testing is required before declaring all in-scope backend functions fully verified or approved.